

API Token Cost Estimation Analysis

<https://gemini.google.com/app/9faa675948afcf55>

User prompt: calculate the est price: `` Pricing Input \$0.15\$0.075/M tokens Output \$0.50\$0.25/M tokens Cache Read \$0.03\$0.015/M tokens `` `` Tokens (selected range) Min 1.27M Max 41.5M Avg 17.6M Sum 88M Latest 11.2M % of Total 100.0% ``

Response: Depending on token type (Input, Output, or Cache Read) and whether standard or discounted rates apply, the estimated cost for **88M tokens** ranges between **\$1.32** and **\$44.00**.

Token Category	Standard Rate	Discounted Rate (50% off)	Total Sum (88M	Latest Request (11.2M	Average (17.6M Tokens)
----------------	---------------	---------------------------	----------------	-----------------------	------------------------

			Tokens)	Tokens)	
Input	\$0.15 / M	\$0.075 / M	\$13.20 (std) / \$6.60 (disc)	\$1.68 (std) / \$0.84 (disc)	\$2.64 (std) / \$1.32 (disc)
Output	\$0.50 / M	\$0.25 / M	\$44.00 (std) / \$22.00 (disc)	\$5.60 (std) / \$2.80 (disc)	\$8.80 (std) / \$4.40 (disc)
Cache Read	\$0.03 / M	\$0.015 / M	\$2.64 (std) / \$1.32 (disc)	\$0.336 (std) / \$0.168 (disc)	\$0.528 (std) / \$0.264 (disc)

Typical Combined Workload Estimates (88M Tokens)

- **80% Input / 20% Output Blend: \$19.36 standard rate (\$9.68 discounted)**
 - **90% Input / 10% Output Blend: \$16.28 standard rate (\$8.14 discounted)**
-

User prompt: what you mean discounted? I send you the price

Response: When copied from platforms like OpenRouter or ZenMux, the original crossed-out price and the active 50% promotional rate paste together into a single text block (e.g., \$0.15\$0.075).

The actual active price is the second, lower rate in each pair:

Active Rates vs. Original

- **Input Tokens: \$0.075 / M ~~(was \$0.15)~~**

- **Output Tokens: \$0.25 / M** ~~(was \$0.50)~~
- **Cache Read Tokens: \$0.015 / M** ~~(was \$0.03)~~

Estimated Cost Breakdown

Token Type	Active Rate	Total Cost (88M Tokens)	Single Request (11.2M Tokens)
100% Input	\$0.075 / M	\$6.60	\$0.84
100% Output	\$0.250 / M	\$22.00	\$2.80
100% Cache Read	\$0.015 / M	\$1.32	\$0.17

Token Type	Active Rate	Total Cost (88M Tokens)	Single Request (11.2M Tokens)
Typical Mix (80% Input / 20% Output)	—	\$9.68	\$1.23

At the active rates, your **88M tokens** will cost **\$6.60** if they are purely input prompts, **\$22.00** if purely model responses, or roughly **\$9.68** for a standard mixed conversation.

User prompt: deepseek/deepseek-v4-flash-0731 vs z-ai/glm-5.3-flash price and coding skill on the openrouter

Response: deepseek/deepseek-v4-flash-0731 provides cheaper raw inference rates and faster generation speeds for isolated coding tasks, while z-ai/glm-5.3-flash delivers a 1.05M token context window and

superior performance for long-horizon agentic workflows across large codebases.

Metric / Feature	DeepSeek V4 Flash (0731)	GLM 5.3 Flash
Input Price (per 1M)	\$0.03 – \$0.08 <i>(provider routing dependent)</i>	\$0.075 <i>(with 50% discount rate)</i>
Output Price (per 1M)	\$0.075 – \$0.18	\$0.25 <i>(with 50% discount rate)</i>
Cache Read (per 1M)	\$0.007 – \$0.016	\$0.015
Context Window	1,000,000 tokens	1,048,576 tokens

Metric / Feature	DeepSeek V4 Flash (0731)	GLM 5.3 Flash
P50 Speed / Throughput	Up to 168 tok/s	~39 tok/s
Best Fit	Fast inline coding, API handlers, quick logic fixes	Autonomous agents, repo-wide multi-file updates

Coding Skill & Workflow Capabilities

DeepSeek V4 Flash (0731)

- **Architecture:** 13B active / 284B total parameter sparse MoE model fine-tuned for high-speed reasoning and tool integration.
- **Coding Strengths:** High benchmark accuracy for single-file syntax, algorithmic logic, test generation, and API integrations.

- **Latency & Speed:** Significantly lower latency and higher generation throughput, making it ideal for real-time IDE completion and fast iteration cycles.

GLM 5.3 Flash (Z.ai)

- **Architecture:** Native multimodal foundation model using hybrid sparse and linear attention for long-context stability.
- **Coding Strengths:** Built specifically for extended agent execution chains (e.g., Claude Code, OpenClaw, persistent developer sub-agents). Handles project-level repository planning, multi-file refactoring across dependencies, and self-correction during long runs.
- **Multimodality:** Can process design mockups, UI wireframes, and architecture diagrams directly into code tasks.

For high-volume, low-latency script writing or bot development, DeepSeek V4 Flash is the more economical option. For persistent agent

workflows that need to analyze full code repositories or follow complex multi-step execution plans, GLM 5.3 Flash provides higher task completion stability.

User prompt: what is the best model on the operouter for persistent agent workflows that need to analyze full code repositories or follow complex multi-step execution plans price and power

Response: For persistent code agents handling large repositories and multi-step execution loops, **Claude 3.7 / 3.5 Sonnet** remains the gold standard for raw reasoning power and tool-calling reliability, while **DeepSeek V4** and **GLM 5.2 / 5.3** offer the best price-to-performance ratios for token-heavy agentic loops.

Model	Input / Output (per 1M)	Cache Read (per 1M)	Context Window	Agentic & Coding Strengths
-------	-------------------------	---------------------	----------------	----------------------------

Claude 3.7 / 3.5 Sonnet

\$3.00 / \$15.00

~\$0.30

200K

Highest Power.
Flawless tool output, zero loop degradation, superior complex diffs and multi-file refactoring.

Model	Input / Output (per 1M)	Cache Read (per 1M)	Context Window	Agentic & Coding Strengths
DeepSeek V4	\$0.14 / \$0.28	~\$0.014	1M	Best Value Powerhouse. High SWE-bench scores, strong logic CoT, 10x–20x cheaper than Sonnet for multi-turn loops.

Model	Input / Output (per 1M)	Cache Read (per 1M)	Context Window	Agentic & Coding Strengths
GLM 5.3 / 5.2 Flash	\$0.075 / \$0.25	\$0.015	1M	Repo-Scale Specialist. Fine-tuned specifically for long-horizon terminal agents, sub-agent execution, and 1M context.
Gemini 3.7 Flash	\$0.10 / \$0.40	~\$0.025	1M	Fast Context Ingestion. Ultra-cheap prompt caching for loading full codebases into memory per step.

Power Breakdown: Which to Choose?

1. Maximum Capability (The "Unattended Agent" Standard)

- **Winner: Claude 3.7 / 3.5 Sonnet**
- **Why:** Persistent agents fail when models drift from tool schemas, hallucinate file paths, or generate invalid git diffs mid-way through a 30-step loop. Sonnet has the lowest instruction-decay rate across long agent chains, making it the top choice for complex architectural planning and critical multi-file changes.

2. Optimal Price & High Volume (The Loop Champion)

- **Winner: DeepSeek V4 / V4-Flash**
- **Why:** Running 50 agent execution steps where each prompt sends 50K–100K tokens of context will quickly cost \$5–\$15 per run on frontier models. DeepSeek V4 drops that same run to \$0.20–\$0.50 while maintaining ~85–90% of Sonnet's coding accuracy.

3. Large Codebase Scanning & Memory

- **Winner: GLM 5.3 Flash or Gemini 3.7 Flash**
- **Why:** When an agent needs to ingest hundreds of repository files at once, 200K context is limiting. Both models feature 1M+ context windows with aggressive prompt caching, allowing agents to retain whole codebases in memory across step transitions at minimal cost.

Recommended Architecture: Two-Tier Routing

To maximize both power and cost efficiency, pair models in your agent framework:

1. **Sub-Agent / Execution Loops (DeepSeek V4 or GLM 5.3):**
Handles file searches, AST parsing, running test suites, reading logs, and writing initial boilerplate patch files.
2. **Orchestrator / Reviewer (Claude Sonnet):** Handles initial system architecture planning, reviews code diffs produced by sub-agents,

and resolves edge cases when a sub-agent gets stuck.

User prompt: what is your definition of Large Codebase

Response: A **large codebase** is any software repository whose physical scale, dependency complexity, and file count exceed what a single developer—or standard LLM context window—can hold in active mental memory at one time.

It is defined across quantitative metrics, team operational limits, and AI context constraints:

Quantitative Scale

Dimension	Medium Codebase	Large Codebase	Enterprise / Monorepo Scale
-----------	-----------------	----------------	-----------------------------

Dimension	Medium Codebase	Large Codebase	Enterprise / Monorepo Scale
Lines of Code (LOC)	20,000 – 100,000	100,000 – 1,000,000+	1,000,000+ to billions
Total Source Files	100 – 500	1,000 – 10,000+	10,000+
LLM Token Footprint	~100K – 400K tokens	1M – 10M+ tokens	20M+ tokens
Team Size	2 – 10 engineers	15 – 100+ engineers	Hundreds of contributors

Key Operational Characteristics

- **Indirect Dependency Coupling:** A change in a shared utility or

database schema produces non-obvious side effects in distant modules without throwing immediate build errors.

- **Fractured Domain Knowledge:** No single engineer understands every subsystem. Onboarding requires dedicated documentation, ownership teams (CODEOWNERS), and strict domain boundaries.
- **Non-Instant Build and Test Cycles:** Compiling, running static analysis, or executing full CI/CD test suites takes minutes to hours rather than seconds.

The AI Developer & LLM Agent Perspective

For AI coding agents (like Claude Code, OpenClaw, or custom sub-agent loops), a codebase crosses into "large" when:

1. **Context Saturation:** The raw source code exceeds the model's single-turn context limit (1M+ tokens), forcing the system to rely on AST (Abstract Syntax Tree) indexing, RAG, file system trees, or sub-agent retrieval routines.

2. **Instruction & Context Decay:** Ingesting 200k+ tokens of raw file context per turn causes the model to hallucinate line numbers, miss imports, or fail to follow system prompts across long execution chains.
 3. **Execution Horizon:** Implementing a single feature requires multi-file git diffs, updating unit tests, adjusting API contracts, and navigating migration scripts across multiple directories.
-

User prompt: does vs code extention cline a reasonable use for price optimization

Response: Yes, using the **Cline** VS Code extension is one of the most effective strategies for LLM price optimization because it gives you granular, turn-by-turn control over API providers, model selection, and prompt caching.

Unlike flat-fee AI extensions (Cursor, GitHub Copilot) or fixed API

interfaces, Cline operates on a Bring-Your-Own-Key (BYOK) architecture, allowing you to route specific agent actions to the cheapest capable model.

Why Cline Works Well for Price Optimization

- **OpenRouter & Prompt Caching Integration:** Cline fully supports OpenRouter's native prompt caching. In persistent agent loops where the system prompt and code context are re-sent every step, caching reduces input token costs by **75% to 90%**.
- **Dynamic Model Switching:** You can instantly swap models mid-project without restarting context. You can use cheap models for code search and cheap edits, then switch to a frontier model only when stuck.
- **Real-Time Cost Transparency:** Displays precise token usage and dollar costs per turn directly in the UI, making high-burn loops visible immediately.

- **Local Model Fallback (Zero Cost):** Connects to local runners (Ollama, LM Studio) to handle boilerplate, documentation, or search tasks for \$0.00.

Recommended Cost-Optimized Strategy in Cline

Task Stage	Recommended OpenRouter Model	Cost Strategy
Repo Scanning & File Search	Gemini 2.5/3.5 Flash or GLM 5.3 Flash	Ingests huge context chunks for fractions of a cent (\$0.075–\$0.10/M).
Complex Diff Generation & Logic	Claude 3.7 / 3.5 Sonnet	Use strictly for core architectural changes or multi-file refactoring.
Terminal Operations,	DeepSeek V4 Flash	Runs fast fix-and-test execution loops for under

Task Stage	Recommended OpenRouter Model	Cost Strategy
Linting & Tests		\$0.05 total.
Basic Refactoring & Docs	Local Model (Ollama / Qwen 2.5)	Zero API cost for non-critical code generation.

Key Pitfalls to Avoid in Cline

- **Unmonitored Auto-Execution:** Because Cline auto-executes terminal commands and reads multiple files, leaving it on a high-cost model (like Sonnet) for 15+ turns can run \$2.00–\$5.00 per task.
- **Disabling Prompt Caching:** Always select OpenRouter endpoints with active prompt caching enabled; disabling it on large context repos will quickly inflate input costs.

User prompt: how can I define cline to use a model for code search and project understanding while using another model for reasoning and another for coding?

Response: Natively, Cline supports a **2-model split** via its built-in **Plan Mode** vs. **Act Mode** configurations. To achieve a true **3-model split** (Search → Reasoning → Coding), you can pair Cline with a local API proxy like **LiteLLM** or use Cline's saved model profiles.

Method 1: Native 2-Model Split (Plan Mode vs. Act Mode)

Cline separates architectural analysis from implementation using two distinct modes in its UI.

Step 1: Configure Plan Mode (Search, Understanding & Reasoning)

1. Open Cline settings and switch to **Plan Mode**.

2. Select your API provider (e.g., OpenRouter).
3. Set the model to a long-context, low-cost model like `google/gemini-2.5-flash` or `z-ai/glm-5.3-flash` (for fast repository search) or `deepseek/deepseek-r1` (for heavy reasoning).

Step 2: Configure Act Mode (Coding & File Execution)

1. Switch to **Act Mode** in Cline settings.
2. Set the model to a dedicated coding model like `anthropic/claude-3.7-sonnet` or `deepseek/deepseek-v3`.

Workflow:

- Ask Cline questions in **Plan Mode** first. It will scan files, parse codebases, build an execution plan, and generate architect decisions for pennies.
- Click "**Switch to Act Mode to execute**"—Cline will pass the

generated plan to your high-tier coding model to write the actual code diffs.

Method 2: True 3-Model Split using LiteLLM Proxy

If you want automated routing where specific tool calls (e.g., `read_file` vs `write_to_file`) run on different models behind the scenes, route Cline through a local **LiteLLM** proxy.

Cline (OpenAI-Compatible) —> LiteLLM Local Proxy (localhost)
├── Search Tools —> Gemini
├── Planning —> DeepSeek
└── Code Writes —> Claude

Step 1: Install and Run LiteLLM

```
pip install litellm
```

Step 2: Create a `config.yaml` for LiteLLM

model_list:

Model 1: Search & Context Ingestion

- model_name: repo-search

litellm_params:

model: openrouter/google/gemini-2.5-flash

api_key: os.environ/OPENROUTER_API_KEY

Model 2: Reasoning & Architecture

- model_name: reasoning-planner

litellm_params:

model: openrouter/deepseek/deepseek-r1

api_key: os.environ/OPENROUTER_API_KEY

Model 3: Code Implementation

- model_name: code-writer

litellm_params:

model: openrouter/anthropic/claude-3.7-sonnet

api_key: os.environ/OPENROUTER_API_KEY

Step 3: Start LiteLLM Proxy

```
export OPENROUTER_API_KEY="your-openrouter-key"  
litellm --config config.yaml --port 4000
```

Step 4: Point Cline to LiteLLM In Cline Settings:

- **API Provider:** OpenAI Compatible
- **Base URL:** `http://localhost:4000/v1`
- **API Key:** anything
- **Model ID:** Choose `repo-search`, `reasoning-planner`, or `code-writer` as needed, or define fallback rules in LiteLLM to dynamically switch endpoints based on context size.

Method 3: Cline Saved Profiles (Manual Quick-Toggle)

Cline allows you to save custom API profiles and swap them with one

click from the bottom header.

1. Create Profile "**1. Context Search**": Set to `z-ai/glm-5.3-flash` or `gemini-2.5-flash`.
2. Create Profile "**2. Reasoning Plan**": Set to `deepseek/deepseek-r1`.
3. Create Profile "**3. Code Writer**": Set to `anthropic/claude-3.7-sonnet` or `deepseek/deepseek-v3`.

Toggle the dropdown in Cline's header as your prompt transitions from repo investigation to architectural strategy, and finally to implementation.

User prompt: can this be achieved using 9router

Response: Yes, **9router** can achieve this setup, and it actually adds built-in optimizations like automatic token compression and failover routing

that make multi-model agent workflows significantly cheaper.

Since 9router exposes a local OpenAI-compliant endpoint (`http://localhost:20128/v1`), it sits between Cline and your model providers to handle model mapping and traffic management.

Step-by-Step Configuration

1. Set Up Model Endpoints in 9router

1. Launch 9router (`npm install -g 9router` and run 9router).
2. Open the 9router Dashboard at `http://localhost:20128`.
3. Connect your API providers (e.g., OpenRouter, Anthropic, Google, DeepSeek) or use 9router's free/cheap tier providers.
4. Note the model identifiers generated by 9router for each role:

- **Search & Understanding:** google/gemini-2.5-flash or z-ai/glm-5.3-flash
- **Reasoning & Planning:** deepseek/deepseek-r1 or deepseek/deepseek-v4
- **Code Writing:** anthropic/claude-3.7-sonnet or deepseek/deepseek-v3

2. Connect Cline to 9router

In Cline Settings:

- **API Provider:** OpenAI Compatible
- **Base URL:** http://localhost:20128/v1
- **API Key:** Paste your local 9router key from the dashboard

3. Map Models in Cline Plan & Act Modes

- **Plan Mode (Search & Reasoning):** Set the model ID to your search or reasoning model (e.g., `google/gemini-2.5-flash` or `deepseek/deepseek-r1`). Cline will use this model to ingest large contexts, search files, read logs, and generate architectural plans.
- **Act Mode (Code Writing):** Set the model ID to your coding model (e.g., `anthropic/claude-3.7-sonnet`). When you approve the plan, Cline switches to this model to execute file changes and writes.

Key Advantages of Using 9router with Cline

- **Built-in Token Compression (RTK):** 9router uses an integrated token killer that automatically intercepts and compresses verbose tool outputs (`git diff`, `grep`, directory listings) by **20% to 40%** before they reach your models, saving significant input costs on large repo scans.

- **3-Tier Automatic Fallback:** If your primary paid model hits a rate limit or quota ceiling mid-task, 9router automatically redirects the request to a cheaper or free tier fallback so your persistent agent session never crashes mid-execution.
 - **Aggregated Usage & Costs:** You get a single dashboard at `localhost:20128` tracking total token burn and dollar expenditure across all connected providers.
-